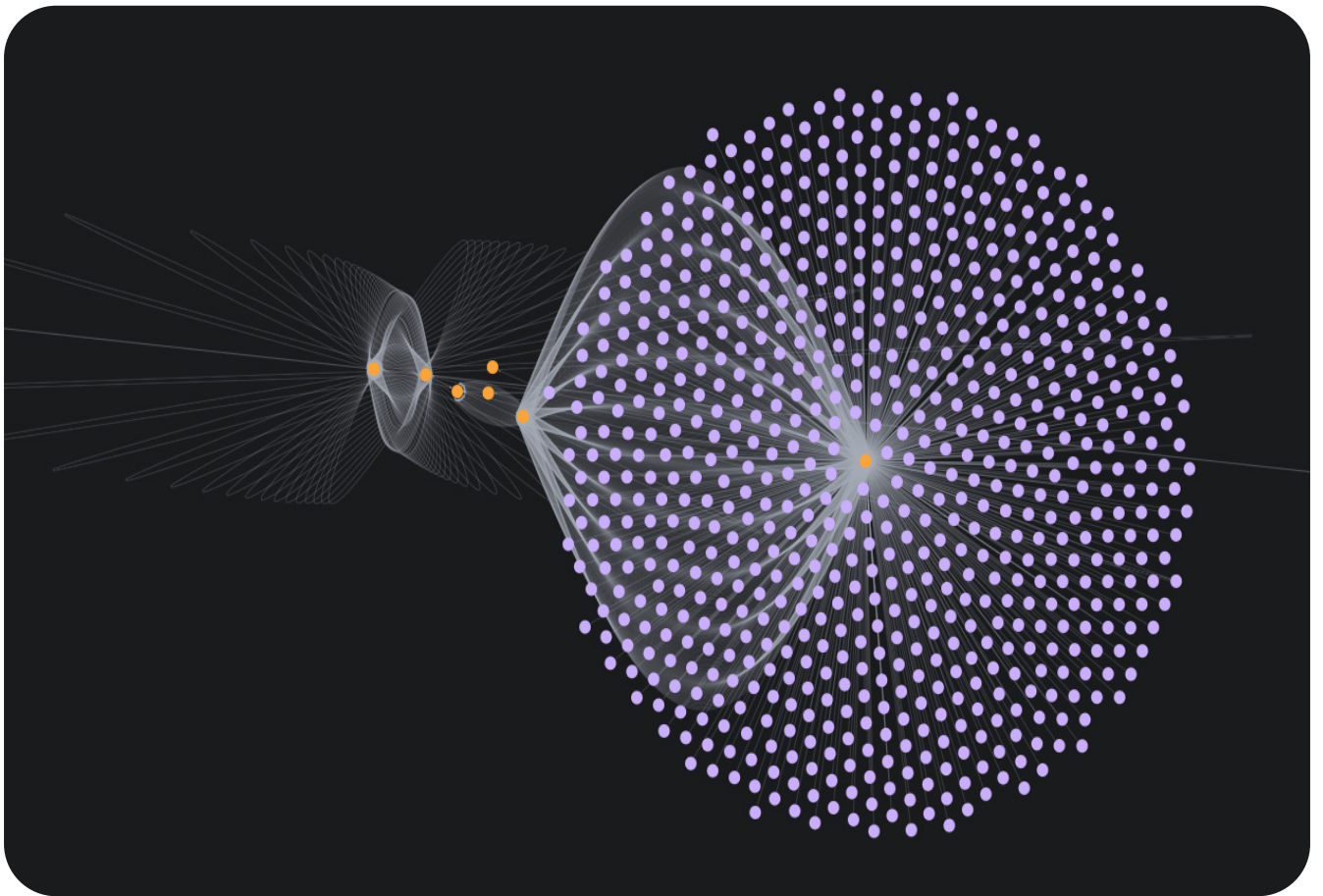

Étude comparative entre le modèle graphe et le modèle relationnel sur la gestion d'un réseau de transport suisse

Rabah ACHOUR
Mohand Arezki BRANECI

13 février 2026



Sous-Graphe du réseau de transport suisse

Résumé

Les réseaux de transport constituent un exemple de données fortement connectées, pour lesquelles une représentation sous forme de graphe est naturelle. Ce projet vise à comparer les versions Cypher 5 et Cypher 25 de Neo4j, en étudiant l'impact des évolutions syntaxiques et sémantiques de Cypher 25 sur l'expressivité des requêtes, leur évaluation et leur consommation mémoire. Les mêmes requêtes, ou des requêtes équivalentes en expressivité, sont également évaluées sur une implémentation relationnelle sous PostgreSQL afin de comparer les stratégies d'exécution. Le rapport met l'accent sur la comparaison des syntaxes Cypher 5 vs Cypher 25 (dont les quantified graph patterns), l'analyse de plans d'exécution, et la discussion des cas où l'une des approches s'évalue bien alors qu'une autre échoue ou consomme significativement plus de ressources.

Table des matières

1	Choix, structuration et création du jeu de données	4
1.1	Jeu de données et contexte	4
1.2	Contraintes d'intégrité	4
1.3	Création des nœuds : Station	4
1.4	Création des nœuds : Trip et import par lots	5
1.5	Association des trajets aux arrêts : relations : HAS_STOP	5
1.6	Sélection d'un sous-graphe d'étude	5
1.7	Construction du graphe de transport : relations : ST_NEXT	6
1.8	Postgres : Création du Schéma Relationnel	6
2	Requêtes simples	9
2.1	Requête croissante	9
2.1.1	Contexte expérimental	9
2.1.2	Cypher 5	9
2.1.3	Cypher 25	9
2.1.4	Postgres	10
2.1.5	Analyse comparative	10
2.1.6	Tableau récapitulatif – Requête croissante	11
2.2	Quantified Graph Patterns (QGP)	12
2.2.1	Contexte expérimental	12
2.2.2	Cypher 5	12
2.2.3	Cypher 25	12
2.2.4	Postgres	13
2.2.5	Tableau récapitulatif – QGP	14
3	Algorithmique de graphes	15
3.1	Construction d'un sous-graphe expérimental contrôlé	15
3.1.1	Phase 1 : Chemins virtuels contrôlés	15
3.1.2	Phase 2 : Allongement massif des chemins	15
3.1.3	Phase 3 : Projection GDS du sous-graphe expérimental	15
3.2	Plus court chemin :	16
3.2.1	Cadre expérimental	16
3.2.2	Plus court chemin non-pondéré	17
3.2.3	Analyse comparative	19
3.2.4	Plus court chemin pondéré	19
3.2.5	Analyse synthétique	22
3.2.6	Tableau comparatif – Chemins virtuels simples	22
3.2.7	Tableau comparatif – Grands chemins virtuels	23
4	Conclusion	24

1 Choix, structuration et création du jeu de données

1.1 Jeu de données et contexte

Le jeu de données utilisé dans ce projet est issu du format **GTFS** (*General Transit Feed Specification*), standard largement utilisé pour décrire des réseaux de transport public.

La source : [GTFS data](#)

Les fichiers GTFS décrivent notamment :

- les arrêts (**stops**),
- les trajets (**trips**),
- la séquence de passage des trajets dans les arrêts (**stop_times**).

L'objectif n'est pas de conserver l'intégralité du modèle GTFS dans Neo4j, mais de construire progressivement un *graphe de transport exploitable* pour l'étude de chemins, tout en gardant une traçabilité claire avec les données sources.

1.2 Contraintes d'intégrité

Avant toute insertion de données, des contraintes d'unicité sont définies afin de garantir l'intégrité du graphe et d'optimiser les opérations de recherche lors des phases d'import.

```
CREATE CONSTRAINT station_id_unique
IF NOT EXISTS
FOR (s:Station)
REQUIRE s.stop_id IS UNIQUE;
```

```
CREATE CONSTRAINT trip_id_unique
IF NOT EXISTS
FOR (t:Trip)
REQUIRE t.trip_id IS UNIQUE;
```

Ces contraintes assurent qu'un arrêt (**:Station**) et qu'un trajet (**:Trip**) ne peuvent être créés qu'une seule fois dans la base.

1.3 Création des nœuds :Station

Les arrêts sont importés à partir du fichier **stops.txt**. Chaque ligne du fichier correspond à un arrêt unique, identifié par **stop_id**.

```
LOAD CSV WITH HEADERS
FROM 'file:///stops.txt'
AS row
MERGE (:Station {
    stop_id: row.stop_id,
    name: row.stop_name,
    lat: toFloat(row.stop_lat),
    lon: toFloat(row.stop_lon)
});
```

L'utilisation de **MERGE** permet d'éviter toute duplication, en cohérence avec la contrainte d'unicité définie précédemment. Les coordonnées géographiques sont conservées afin de permettre, dans les sections ultérieures, l'étude d'algorithmes de plus court chemin tenant compte de distances ou de coûts.

1.4 Création des nœuds :Trip et import par lots

Les trajets GTFS sont ensuite importés à partir du fichier `trips.csv`. Chaque trajet est représenté par un nœud `:Trip` identifié par `trip_id`.

```
LOAD CSV WITH HEADERS
FROM 'file:///trips.csv'
AS row
CALL (row) {
    MERGE (:Trip {trip_id: row.trip_id})
} IN TRANSACTIONS OF 10000 ROWS;
```

Traitement par lots (batch). L'option `IN TRANSACTIONS OF 10000 ROWS` est essentielle compte tenu du volume important de données. Elle permet :

- de limiter la taille des transactions,
- de réduire la consommation mémoire,
- d'éviter des échecs dus à des transactions trop volumineuses.

Cette stratégie est systématiquement adoptée pour les imports massifs dans la suite du projet.

1.5 Association des trajets aux arrêts : relations :HAS_STOP

Le fichier `stop_times.csv` décrit la séquence ordonnée des arrêts pour chaque trajet. Cette information est modélisée par une relation intermédiaire `:HAS_STOP` entre un trajet et une station.

```
LOAD CSV WITH HEADERS
FROM 'file:///stop_times.csv'
AS row
CALL (row) {
    MATCH (t:Trip {trip_id: row.trip_id})
    MATCH (s:Station {stop_id: row.stop_id})
    MERGE (t)-[:HAS_STOP {
        arrival_time: row.arrival_time,
        departure_time: row.departure_time,
        stop_sequence: toInteger(row.stop_sequence)
    }]->(s)
} IN TRANSACTIONS OF 10000 ROWS;
```

À ce stade, le graphe reflète fidèlement la structure GTFS : les trajets sont reliés aux stations, avec un ordre de passage explicite (`stop_sequence`), mais aucune relation directe n'existe encore entre deux stations consécutives.

1.6 Sélection d'un sous-graphe d'étude

Afin de rendre les expérimentations exploitables d'un point de vue computationnel, un sous-ensemble de trajets est volontairement sélectionné.

```
MATCH (t:Trip)
WITH t
LIMIT 500
SET t:StudyTrip;
```

Les trajets marqués par le label `:StudyTrip` constituent le *sous-graphe d'étude*. Cette étape est cruciale pour :

-
- limiter l’explosion combinatoire lors des requêtes de chemins,
 - conserver un graphe représentatif tout en restant analysable,
 - permettre une comparaison réaliste entre Cypher 5, Cypher 25 et GDS.

1.7 Construction du graphe de transport : relations :ST_NEXT

La relation :ST_NEXT représente le cœur du graphe de transport : elle relie deux stations consécutives au sein d’un même trajet.

À partir des trajets sélectionnés (:StudyTrip), les relations :ST_NEXT sont construites comme suit :

```
MATCH (t:Trip:StudyTrip)
WITH t.trip_id AS tid
MATCH (a:Station)-[r:NEXT]->(b:Station)
WHERE r.trip_id = tid
MERGE (a)-[x:ST_NEXT {trip_id: tid}]->(b)
SET x.duration = r.duration;
```

Chaque relation :ST_NEXT porte :

- l’identifiant du trajet (trip_id),
- une durée (duration) utilisée comme poids dans les algorithmes de plus court chemin.

Cheminement conceptuel : Le processus de construction peut être résumé ainsi :

Création de Trip → Création de HAS_STOP → Création de ST_NEXT

Autrement dit, les relations :ST_NEXT constituent une *projection orientée transport* du modèle GTFS initial, dans laquelle les stations deviennent directement connectées selon l’ordre réel de circulation.

Résultat : À l’issue de ces étapes, on obtient un graphe de stations reliées par des relations directionnelles pondérées, prêt à être exploité dans les expériences comparatives Cypher 5 / Cypher 25 / GDS présentées dans la suite du rapport.

1.8 Postgres : Création du Schéma Relationnel

La première étape a consisté à définir une structure de base de données robuste respectant les contraintes d’intégrité référentielle et optimisée pour les calculs de chemins :

- **Table stops :** Répertoire l’ensemble des stations.
- **Table trips :** Répertoire les trajets uniques du réseau, permettant de lier les horaires aux services de transport.
- **Table stop_times :** Contient les horaires de passage.
- **Table study_trips :** Table pivot utilisée pour isoler un échantillon de 500 trajets. Cette réduction stratégique du jeu de données est essentielle pour garantir la terminaison des requêtes récursives complexes et éviter l’explosion combinatoire.
- **Table st_next :** Le rôle de cette table est de transformer une structure tabulaire événementielle en une structure de graphe orientée arêtes (*d’où vient le train et où va-t-il*).
 - **Abstraction des relations :** Contrairement à la table stop_times qui liste des arrêts isolés, st_next matérialise explicitement les arcs du graphe en liant une station source (source_stop) à une station cible (target_stop).
 - **Pondération des arêtes :** Elle stocke la propriété duration (type INTERVAL), permettant de calculer des chemins les plus courts basés sur le temps réel plutôt que sur le simple nombre d’arrêts.

Création des tables

```
-- Table des arrêts (Stops)
CREATE TABLE stops (
    stop_id TEXT PRIMARY KEY,
    stop_name TEXT,
    stop_lat DOUBLE PRECISION,
    stop_lon DOUBLE PRECISION,
    location_type TEXT,
    parent_station TEXT,
    platform_code TEXT,
    original_stop_id TEXT
);

-- Table des trajets (Trips)
CREATE TABLE trips (
    route_id TEXT,
    service_id TEXT,
    trip_id TEXT PRIMARY KEY,
    trip_headsign TEXT,
    trip_short_name TEXT,
    direction_id INTEGER,
    block_id TEXT,
    original_trip_id TEXT,
    hints TEXT
);

-- Table des Stop Times
CREATE TABLE stop_times (
    trip_id TEXT REFERENCES trips(trip_id),
    arrival_time TEXT,
    departure_time TEXT,
    stop_id TEXT REFERENCES stops(stop_id),
    stop_sequence INTEGER,
    pickup_type INTEGER,
    drop_off_type INTEGER,
    PRIMARY KEY (trip_id, stop_sequence)
);

-- Table de jonction pour le graphe (ST_NEXT)
CREATE TABLE st_next (
    source_stop TEXT REFERENCES stops(stop_id),
    target_stop TEXT REFERENCES stops(stop_id),
    trip_id TEXT REFERENCES trips(trip_id),
    duration INTERVAL
);

\begin{verbatim}
CREATE TABLE study_trips AS
SELECT trip_id
```

```
FROM trips
LIMIT 500;
```

```
INSERT INTO st_next (source_stop, target_stop, trip_id, duration)
SELECT
    s1.stop_id AS source_stop,
    s2.stop_id AS target_stop,
    s1.trip_id,
    (s2.arrival_time::time - s1.departure_time::time) AS duration
FROM stop_times s1
JOIN stop_times s2 ON s1.trip_id = s2.trip_id
    AND s2.stop_sequence = s1.stop_sequence + 1
JOIN study_trips st ON s1.trip_id = st.trip_id;

-- Importation depuis tes fichiers CSV (ajuste les chemins)
COPY stops FROM '/chemin/vers/stops.txt' WITH (FORMAT csv, HEADER true);
COPY trips FROM '/chemin/vers/trips.txt' WITH (FORMAT csv, HEADER true);
COPY stop_times FROM '/chemin/vers/stop_times.txt' WITH (FORMAT csv, HEADER true);
```

Requête Récursive SQL : Trouver tous les arrêts accessibles (atteignables) depuis l'arrêt '1100008'.

```
EXPLAIN (ANALYZE, COSTS, BUFFERS, VERBOSE)
WITH RECURSIVE reachability(stop_id, path, depth) AS (

    SELECT
        stop_id,
        ARRAY[stop_id]::text[],
        1
    FROM stop_times
    WHERE stop_id = '1100008'

    UNION ALL

    SELECT
        st.stop_id,
        (r.path || st.stop_id)::text[],
        r.depth + 1
    FROM stop_times st
    JOIN reachability r ON st.stop_id <> ALL(r.path)
    WHERE r.depth < 3 -- Limite pour éviter le Time-out
)
SELECT DISTINCT stop_id FROM reachability;
```

Temps d'exécution excessif (35 minutes) - requête annulée par l'utilisateur.

2 Requêtes simples

Cette section comporte :

- comparaison de requêtes Cypher 5 et Cypher 25, y compris la réécriture de requêtes problématiques,
- inclusion de plans d'exécution (**EXPLAIN** et **PROFILE**),
- traduction SQL (incluant au moins une requête SQL récursive) et comparaison avec Cypher,
- étude d'une requête utilisant les quantified graph patterns,
- étude d'une requête filtrant des chemins selon une propriété d'arête croissante (avec **NOT EXISTS** en Cypher 5 et **allReduce** en Cypher 25).

2.1 Requête croissante

2.1.1 Contexte expérimental

Le graphe d'étude contient :

- 67 898 nœuds **:Station**
- 3 107 relations **:ST_NEXT**

L'objectif est de compter les chemins de longueur 2 tels que :

- les deux relations successives appartiennent au même **trip_id**,
- la propriété **duration** est strictement croissante.

Il s'agit donc d'un filtrage sur propriétés d'arêtes consécutives.

2.1.2 Cypher 5

Requête

PROFILE

```
MATCH p = (s:Station)-[rels:ST_NEXT*2]->(t:Station)
```

WHERE

```
  NOT EXISTS {
    WITH rels
    UNWIND range(0, size(rels)-2) AS i
    WITH rels[i] AS r1, rels[i+1] AS r2
    WHERE r1.trip_id <> r2.trip_id
      OR r1.duration >= r2.duration
    RETURN 1
  }
```

```
RETURN count(p);
```

Résultats :

- Total database accesses : 11 971 151
- Mémoire allouée : 308 776 bytes
- Temps total : 3 minutes 13 secondes

L'opérateur dominant est **VarLengthExpand(A11)**. Environ 44 millions de chemins candidats sont générés avant filtrage complet.

2.1.3 Cypher 25

Requête

CYPHER 25

PROFILE

```

MATCH p = (s:Station)-[rels:ST_NEXT*2]->(t:Station)
WHERE
    allReduce(
        ok = true,
        i IN range(0, size(rels)-2) |
        ok AND
        rels[i].trip_id = rels[i+1].trip_id AND
        rels[i].duration < rels[i+1].duration,
        ok
    )
RETURN count(p);

```

Résultats :

- Total database accesses : 9 714 517
- Mémoire allouée : 624 bytes
- Temps total : 3 minutes 8 secondes

2.1.4 Postgres

Requête

```

EXPLAIN (ANALYZE, BUFFERS, VERBOSE)
SELECT count(*)
FROM st_next sn1
JOIN st_next sn2 ON sn1.target_stop = sn2.source_stop
WHERE sn1.trip_id = sn2.trip_id
    AND sn1.duration <= sn2.duration

```

Résultats :

- Mémoire allouée : 1351kB
- Temps total : 225.875 ms

2.1.5 Analyse comparative

Les deux versions génèrent un volume intermédiaire similaire, car la complexité est dominée par :

VarLengthExpand(All)

- La performance en temps est équivalente
 - le coût principal provient de l'expansion,
 - le filtrage intervient après génération.
- En revanche, une différence majeure apparaît sur la mémoire :
 - Cypher 5 : 308 776 bytes
 - Cypher 25 : 624 bytes

Pourquoi ?

- Cypher 5 utilise `NOT EXISTS + UNWIND`, ce qui génère des structures intermédiaires (anti-join + projections temporaires).
- Cypher 25 encapsule la logique dans `allReduce`, évaluée de manière incrémentale sans matérialisation explicite des paires d'arêtes.

Ainsi, le coût en temps est dominé par l'expansion, mais l'empreinte mémoire dépend fortement de la structure interne du filtre.

Postgres

- PostgreSQL traite la requête comme une simple jointure de deux tables indexées. Grâce aux index B-Tree il élimine les candidats invalides en quelques millisecondes.
- La mémoire allouée par Postgres correspond à celle nécessaire pour créer la table de hachage ou le tri nécessaire à la jointure. Bien que plus élevée en apparence, elle reste négligeable pour un serveur moderne.

2.1.6 Tableau récapitulatif – Requête croissante

Critère	Cypher 5	Cypher 25	Postgres
Formulation du filtre	NOT EXISTS + UNWIND	allReduce	Join
Database accesses	11 971 151	9 714 517	NA
Mémoire allouée	308 KB	624 B	1351 kB
Temps total	3 min	3 min	225 ms

TABLE 1 – Comparaison des performances pour la requête croissante

2.2 Quantified Graph Patterns (QGP)

2.2.1 Contexte expérimental

Le graphe d'étude contient :

- 67 898 nœuds :`Station`
- 3 107 relations :`ST_NEXT`

L'objectif de cette expérience est d'évaluer les performances d'une requête utilisant un motif de longueur variable (2 à 3 relations successives) et de comparer :

- l'écriture classique en Cypher 5 avec un pattern variable,
- l'écriture en Cypher 25 utilisant un Quantified Graph Pattern (QGP),
- le comportement du planificateur et du moteur d'exécution.

La requête cherche, pour chaque station source `s`, le nombre de chemins de longueur comprise entre 2 et 3 via des relations :`ST_NEXT`, tout en garantissant que les nœuds intermédiaires sont bien des :`Station`.

2.2.2 Cypher 5

Requête

```
MATCH p = (s:Station)-[:ST_NEXT*2..3]->(t)
WHERE ALL(n IN nodes(p)[1..-1] WHERE n:Station)
RETURN s, count(p) AS nb;
```

Résultats :

- Accès base de données : 2 261 032 492
- Mémoire allouée : 349 136 bytes
- Temps total : 3 minutes 33 secondes

Le plan montre une opération dominante de type `VarLengthExpand(All)` générant plus de 452 millions de chemins candidats. Le filtrage via `ALL(nodes(p) ...)` intervient après construction du chemin, ce qui entraîne un volume intermédiaire massif. L'agrégation ordonnée (`OrderedAggregation`) contribue également au coût global.

2.2.3 Cypher 25

Requête

```
CYPHER 25
MATCH (s:Station)
MATCH (s)-[:ST_NEXT]->(:Station)
      ((:Station)-[:ST_NEXT]->(:Station)){1,2}
RETURN DISTINCT s;
```

Résultats :

- Accès base de données : 2 285 368 658
- Mémoire allouée : 264 965 952 bytes
- Temps total : 2 minutes

Le plan met en évidence l'opérateur `Repeat(All, Trail)`, spécifique aux patterns quantifiés. Contrairement à la version Cypher 5, la structure du motif est explicitement encodée dans le pattern, ce qui permet une exécution plus structurée.

Le temps d'exécution est significativement inférieur (3 min 12 s contre 5 min 30 s), mais la mémoire consommée est nettement plus élevée (13 MB contre 34 KB). Cette différence provient du mécanisme interne de gestion des répétitions et du maintien des structures intermédiaires nécessaires au `Repeat(All, Trail)`.

2.2.4 Postgres

Requête

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS, SUMMARY)
WITH RECURSIVE parcours AS (
    SELECT
        source_stop AS station_depart,
        target_stop AS station_actuelle,
        1 AS longueur
    FROM st_next

    UNION ALL

    SELECT
        p.station_depart,
        n.target_stop,
        p.longueur + 1
    FROM st_next n
    INNER JOIN parcours p ON n.source_stop = p.station_actuelle
    WHERE p.longueur < 3
)

SELECT
    station_depart,
    count(*) AS nb
FROM parcours
WHERE longueur IN (2, 3)
GROUP BY station_depart;
```

Résultats :

- Lignes traitées : 452 724 016 lignes (chemins).
- Mémoire allouée : 17 038 360 kB 17 Go
- Temps total : 8 minutes 45 secondes

Explication : L'analyse du plan d'exécution PostgreSQL révèle les limites du moteur relationnel face à l'explosion combinatoire des chemins. Pour une recherche récursive sans filtre restrictif, le système a généré plus de **452 millions de lignes**.

PostgreSQL a dû déporter le calcul sur le stockage temporaire, consommant **17 Go de disque**. Ce passage du mode 'In-Memory' au mode 'Disk' fait grimper le temps d'exécution à plus de **8 minutes**, illustrant parfaitement le concept d'intractabilité décrit par Gheerbrant et al. lorsque la sémantique de chemin n'est pas optimisée

2.2.5 Tableau récapitulatif – QGP

Critère	Cypher 5	Cypher 25	Postgres
Accès BDD	2 Bil	2 Bil	NA
Mémoire allouée	349 KB	265 MB	17 Go
Temps total	3 min 33 s	2 min	8 min 45

TABLE 2 – Comparaison sur les Quantified Graph Patterns

3 Algorithmique de graphes

3.1 Construction d'un sous-graphe expérimental contrôlé

L'analyse du graphe GTFS initial a révélé une topologie fortement centralisée, avec un diamètre faible et des chemins relativement courts. Cette structure ne permettait pas d'observer clairement les différences de comportement entre requêtes déclaratives (Cypher) et algorithmes spécialisés (GDS).

Afin d'étudier ces différences dans un cadre contrôlé, nous avons construit un sous-graphe expérimental composé de chemins artificiellement allongés reliant deux stations spécifiques : LONG_A (source) et LONG_B (cible).

3.1.1 Phase 1 : Chemins virtuels contrôlés

Cinq chemins linéaires concurrents ont été créés entre LONG_A et LONG_B, de longueurs :

997, 998, 999, 1000, 1001

Chaque chemin est constitué exclusivement de relations :ST_NEXT, sans branchement intermédiaire. Les stations intermédiaires portent un identifiant Pxxx_i, permettant de filtrer explicitement ces chemins via la condition :

stop_id STARTS WITH 'P'

Cette première phase permet de valider la correction fonctionnelle des requêtes de plus court chemin.

3.1.2 Phase 2 : Allongement massif des chemins

Afin d'examiner les limites des approches déclaratives, nous avons allongé les cinq chemins de la Phase 1 en insérant un grand nombre de nœuds intermédiaires entre chaque paire de stations consécutives.

Chaque relation initiale :

$\text{Station}_i \rightarrow \text{Station}_{i+1}$

est remplacée par une chaîne linéaire composée de dizaines de segments supplémentaires. Les longueurs deviennent alors :

$997 \times 91, 998 \times 90, 999 \times 94, 1000 \times 88, 1001 \times 97$

relations :ST_NEXT respectivement.

La topologie reste strictement linéaire (cinq chemins concurrents, sans bifurcation), mais la profondeur effective atteint désormais plusieurs dizaines de milliers d'arêtes.

Les durées associées aux segments intermédiaires sont générées aléatoirement afin d'imposer un véritable calcul pondéré.

Cette configuration constitue un *stress test* destiné à révéler les limites structurelles de l'énumération déclarative des chemins.

3.1.3 Phase 3 : Projection GDS du sous-graphe expérimental

Afin d'exécuter les algorithmes de la bibliothèque Graph Data Science (GDS), nous construisons une projection en mémoire du sous-graphe expérimental :

```
CALL gds.graph.project.cypher(
  'long_P_paths',
  ,
  MATCH (n:Station)
  WHERE
    n.stop_id = "LONG_A"
    OR n.stop_id = "LONG_B"
    OR n.stop_id STARTS WITH "P997_"
    OR n.stop_id STARTS WITH "P998_"
    OR n.stop_id STARTS WITH "P999_"
    OR n.stop_id STARTS WITH "P1000_"
    OR n.stop_id STARTS WITH "P1001_"
  RETURN id(n) AS id,
         n.lat_sim AS lat,
         n.lon_sim AS lon
  ,
  ,
  MATCH (a:Station)-[r:ST_NEXT]->(b:Station)
  WHERE r.trip_id IN
    ["P997","P998","P999","P1000","P1001","ENTRY","EXIT"]
  RETURN id(a) AS source,
         id(b) AS target,
         coalesce(r.duration, 1.0) AS duration
  ,
);
```

Cette projection :

- isole uniquement les cinq chemins virtuels et les nœuds LONG_A et LONG_B,
- conserve la propriété **duration** comme poids,
- stocke les coordonnées simulées (**lat_sim**, **lon_sim**) pour les expériences ultérieures avec A*.

Les algorithmes GDS s'exécutent ensuite sur ce graphe en mémoire, optimisé pour le calcul algorithmique, et distinct du graphe transactionnel natif de Neo4j.

3.2 Plus court chemin :

3.2.1 Cadre expérimental

Nous calculons le plus court chemin entre LONG_A et LONG_B dans le sous-graphe des grands chemins virtuels (Phase 2).

Deux variantes sont étudiées :

- **Non-pondéré** : minimisation du nombre de sauts.
- **Pondéré** : minimisation de la somme des **duration**.

Projection GDS : Le graphe projeté **long_P_paths** contient :

- **468 607 nœuds**
- **1 375 904 relations**
- Densité : 6.26×10^{-6}

Mémoire occupée par la projection : **88 MiB** (92 873 016 bytes).

Cette mémoire est allouée une seule fois pour l'ensemble des algorithmes GDS exécutés sur ce graphe.

3.2.2 Plus court chemin non-pondéré

Nous comparons ici les différentes implémentations du plus court chemin non pondéré entre LONG_A et LONG_B sur les grands chemins virtuels (Phase 2).

Dans ce cadre, la distance correspond uniquement au nombre de sauts (arêtes).

Cypher 5 – shortestPath()

```
MATCH p = shortestPath(
  (a:Station {stop_id:'LONG_A'})-[:ST_NEXT*1..]->
  (b:Station {stop_id:'LONG_B'})
)
RETURN p, length(p);
```

La fonction `shortestPath()` implémente un **BFS bidirectionnel** :

- Taille du plus court chemin : 89322
 - une exploration depuis la source,
 - une exploration simultanée depuis la cible,
 - arrêt dès que les deux fronts se rencontrent.
- où b est le facteur de branchement et d la distance entre les deux nœuds.

Résultats expérimentaux :

- Temps total : 1 122 ms
- Mémoire allouée : 18 876 800 bytes (18 MB)
- Accès base : 2 397 382

Cypher 25 – SHORTEST 1

```
CYPHER 25
MATCH p = SHORTEST 1
  (a:Station {stop_id:'LONG_A'})-[:ST_NEXT]->{1,}
  (b:Station {stop_id:'LONG_B'})
RETURN p, length(p);
```

L'opérateur `SHORTEST 1` repose également sur un **BFS bidirectionnel optimisé**, garantissant la minimalité du chemin retourné.

Résultats :

- Taille du plus court chemin : 89322
- Temps total : 1 237 ms
- Mémoire allouée : 18 876 800 bytes (18 MB)
- Accès base : 2 487 203

Complexité théorique du BFS bidirectionnel :

$$O(b^{d/2})$$

avec b le nombre moyen de voisins par nœud du graphe et d le nombre d'arêtes du plus court chemin.

Résultats :

- Temps total : 1 237 ms
- Mémoire allouée : 18 876 800 bytes (18 MB)
- Accès base : 2 487 203

Postgres

```
EXPLAIN ANALYZE
WITH RECURSIVE find_shortest AS (
    SELECT source_stop, target_stop, 1 AS depth
    FROM st_next WHERE source_stop = 'LONG_A'
    UNION ALL
    SELECT n.source_stop, n.target_stop, f.depth + 1
    FROM st_next n
    JOIN find_shortest f ON n.source_stop = f.target_stop
    WHERE
        f.target_stop <> 'LONG_B'
)
SELECT depth FROM find_shortest WHERE target_stop = 'LONG_B'
ORDER BY depth ASC LIMIT 1;

— Temps total : 0.288 ms
— Mémoire allouée : 299kB
```

GDS – BFS unidirectionnel

```
PROFILE
MATCH (a:Station {stop_id:'LONG_A'}),
      (b:Station {stop_id:'LONG_B'})
CALL gds.bfs.stream(
    'long_P_paths',
    {
        sourceNode: id(a),
        targetNodes: [id(b)]
    }
)
YIELD path
RETURN length(path);
```

L'algorithme `gds.bfs.stream` est un **BFS unidirectionnel** :

- exploration depuis la source uniquement,
- propagation couche par couche,
- arrêt lorsqu'un nœud cible est rencontré.

Contrairement à `shortestPath()`, il s'agit d'un algorithme de traversal général, non spécifiquement optimisé pour le calcul du plus court chemin entre deux nœuds.

Le resultat retourné n'est généralement pas le plus court chemin

Complexité théorique :

$$O(b^d)$$

avec b le nombre moyen de voisins par noeud du graphe et d le nombre d'arêtes du plus court chemin.

Résultats expérimentaux :

- Taille du chemin trouvé : **449109** >> BFS bidirectionnel (89322)
- Temps total : 1 516 ms

— Mémoire allouée : 312 bytes

La mémoire principale consommée (88 MiB) correspond à la projection GDS, indépendante de l'exécution de la requête.

GDS – Dijkstra non-pondéré

```
CALL gds.shortestPath.dijkstra.stream(
  'long_P_paths',
  {
    sourceNode: id(a),
    targetNode: id(b)
  }
)
YIELD totalCost, path
RETURN totalCost, path;
```

En l'absence de poids, Dijkstra se comporte comme un BFS mais avec une structure de priorité interne.

Complexité théorique :

$$O(E \log V)$$

avec V le nombre de noeuds du graphe et E le nombre d'arêtes du graphe

Résultats :

- Temps total : 5 395 ms
- Mémoire requête : 312 bytes

3.2.3 Analyse comparative

- Les implémentations Cypher 5 et 25 utilisent un BFS bidirectionnel optimisé, ce qui réduit exponentiellement l'espace exploré.
- Le BFS unidirectionnel de GDS explore un espace potentiellement plus large ($O(b^d)$) et ne retourne pas le plus court chemin.
- Dijkstra est plus général, mais introduit un surcoût algorithmique inutile en non-pondéré.
- La mémoire observée en Cypher (18 MB) correspond aux structures internes du plan.
- En GDS, la mémoire de requête est minimale, mais la projection (88 MiB) constitue un coût fixe préalable.

3.2.4 Plus court chemin pondéré

Cypher 5 :

```
MATCH p =
  (a:Station {stop_id:'LONG_A'})-[:ST_NEXT*1..]->
  (b:Station {stop_id:'LONG_B'})
WHERE ALL(n IN nodes(p)[1..-1]
  WHERE left(n.stop_id,4) IN
    ['P997','P998','P999','P1000','P1001'])
WITH p,
  reduce(cost = 0.0,
    r IN relationships(p) | cost + r.duration) AS totalCost
RETURN length(p) AS hops, totalCost
```

```
ORDER BY totalCost
LIMIT 1;
```

Cette requête :

- énumère tous les chemins entre la source et la cible,
- restreint l'espace aux cinq chemins virtuels via `ALL`,
- calcule le coût total par `reduce`,
- sélectionne le minimum par tri global.

Cypher 25 :

```
CYPHER 25
MATCH p =
  (a:Station {stop_id:'LONG_A'})-[:ST_NEXT]->{1,}
  (b:Station {stop_id:'LONG_B'})
WHERE allReduce(
  ok = true,
  n IN nodes(p)[1..-1] |
  ok AND left(n.stop_id,4) IN
    ['P997','P998','P999','P1000','P1001'],
  ok
)
WITH p,
  reduce(cost = 0.0,
    r IN relationships(p) | cost + r.duration) AS totalCost
RETURN length(p) AS hops, totalCost
ORDER BY totalCost
LIMIT 1;
```

La version Cypher 25 exprime la même contrainte à l'aide de `allReduce`, qui accumule un booléen sur l'ensemble des nœuds intermédiaires. La stratégie reste identique : *énumération complète suivie d'une minimisation globale*.

Résultat expérimental (grands chemins virtuels). Les deux requêtes ont provoqué :

```
Failed to allocate memory in a memory pool
(dbms.memory.transaction.total.max exceeded)
```

Le moteur tente de matérialiser des chemins contenant plusieurs dizaines de milliers d'arêtes avant la sélection du minimum, ce qui entraîne une explosion mémoire.

Postgres Les résultats de la requête sont faits sur **les chemins virtuels simples**

```
EXPLAIN (ANALYZE, BUFFERS, VERBOSE, SETTINGS, SUMMARY)
WITH RECURSIVE dijkstra AS (
  SELECT
    target_stop,
    duration as total_cost,
    1 AS depth,
    ARRAY[source_stop::text, target_stop::text] AS path
  FROM st_next
  WHERE source_stop = 'LONG_A'
```

```

UNION ALL

SELECT
    sn.target_stop,
    (EXTRACT(EPOCH FROM d.total_cost) +
     EXTRACT(EPOCH FROM sn.duration) || ' seconds')::interval as total_cost,
    d.depth + 1,
    d.path || sn.target_stop::text
FROM st_next sn
JOIN dijkstra d ON sn.source_stop = d.target_stop
WHERE d.total_cost < interval '20 minutes'
      AND d.depth < 1005
      AND NOT sn.target_stop::text = ANY(d.path)
)
SELECT
    total_cost,
    depth as nb_segments,
    path as itinerary
FROM dijkstra
WHERE target_stop = 'LONG_B'
ORDER BY total_cost ASC
LIMIT 1;

```

NB : Les résultats de cette requête sont faits sur les **chemins virtuels simples**

Résultats :

- Temps total : 305.873 ms
- Mémoire allouée : 32,07 MiB

L'analyse des résultats montre que, dans notre configuration expérimentale linéaire, PostgreSQL obtient d'excellentes performances pour le calcul du plus court chemin entre deux points spécifiques. La requête repose sur une CTE récursive, qui explore progressivement les voisins à profondeur croissante. Grâce à la faible branchement du graphe expérimental et à l'efficacité des index B-Tree pour les jointures, PostgreSQL limite le volume intermédiaire généré. Toutefois, cette performance dépend fortement de la structure du graphe : en présence d'un grand nombre d'arêtes, la récursion peut entraîner une explosion combinatoire, comme observé dans les expériences précédentes (QGP).

GDS – Dijkstra pondéré :

```

CALL gds.shortestPath.dijkstra.stream(
    'long_P_paths',
    {
        sourceNode: id(a),
        targetNode: id(b),
        relationshipWeightProperty: 'duration'
    }
)
YIELD totalCost, path
RETURN totalCost, path;

```

Résultats :

- Temps total : 5 899 ms
- Mémoire requête : 312 bytes
- Mémoire projection : 88 MB

3.2.5 Analyse synthétique

- Les versions pondérées en Cypher 5 et 25 reposent sur une minimisation après énumération, ce qui devient non scalable sur des chemins très profonds.
- L'introduction de `allReduce` en Cypher 25 améliore l'expressivité, mais ne change pas la nature algorithmique du traitement.
- Dijkstra (GDS) repose sur une relaxation incrémentale et reste stable en mémoire et en temps, la mémoire principale étant celle de la projection (88 MB).

3.2.6 Tableau comparatif – Chemins virtuels simples**Graphe transactionnel (Cypher 5 / 25) :**

- Nombre de nœuds : **109 480**
- Nombre de relations : **9 107**

Projection GDS (long_P_paths) :

- Nombre de nœuds : **109 480**
- Nombre de relations : **9 107**
- Mémoire occupée : **312 B**

Configuration	Cypher 5	Cypher 25	GDS	Postgres
Non-pondéré				
Temps (s)	0.005	0.003	0.01	0.00026
Mémoire requête	~297 KB	~297 KB	30 B	299 KB
Pondéré				
Temps (s)	0.8	0.2	0.02	0.30
Mémoire requête	–	–	30 B	33 MB

TABLE 3 – Plus court chemin sur chemins virtuels simples

3.2.7 Tableau comparatif – Grands chemins virtuels

Graphe transactionnel (Cypher 5 / 25) :

- Nombre de nœuds : **572 095**
- Nombre de relations : **1 379 011**

Projection GDS (long_P_paths) :

- Nombre de nœuds : **468 607**
- Nombre de relations : **1 375 904**
- Mémoire occupée : **88 MB**

Configuration	Cypher 5	Cypher 25	GDS
Non-pondéré			
Temps (s)	1.12	1.24	5.40
Mémoire requête	~18 MB	~18 MB	312 B
Pondéré			
Temps (s)	Memory Out	Memory Out	5.90
Mémoire requête	–	–	312 B

TABLE 4 – Plus court chemin sur grands chemins virtuels

4 Conclusion

Cette étude comparative met en évidence que les améliorations apportées par Cypher 25 (Quantified Graph Patterns et `allReduce`) améliorent nettement l'expressivité et peuvent réduire l'empreinte mémoire pour certains filtres complexes, sans toutefois modifier la nature déclarative du moteur, fondée sur l'énumération de chemins. Sur des graphes peu profonds, Cypher 5 et Cypher 25 offrent des performances comparables, tandis que PostgreSQL excelle sur des jointures structurées. En revanche, pour les calculs de plus court chemin pondéré sur des graphes profonds, les approches déclaratives atteignent leurs limites mémoire, alors que les algorithmes spécialisés de GDS (comme Dijkstra) restent stables grâce à une stratégie algorithmique incrémentale. Ces résultats montrent que le choix entre modèle graphe transactionnel, modèle relationnel et bibliothèque algorithmique dépend avant tout de la nature structurelle du problème traité.